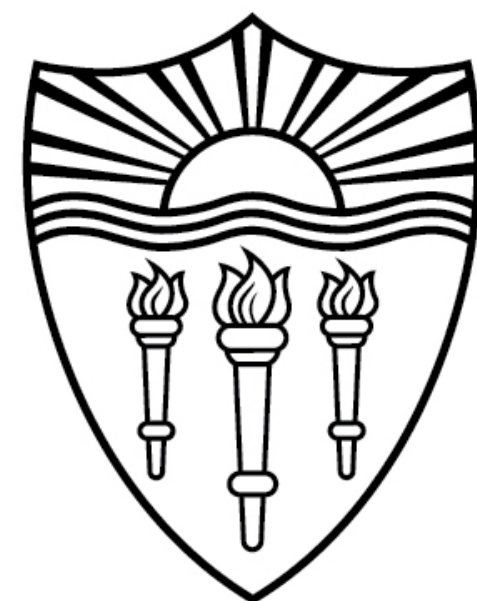


CSCI 544: Applied Natural Language Processing

Seq2seq Generation & Neural Machine Translation

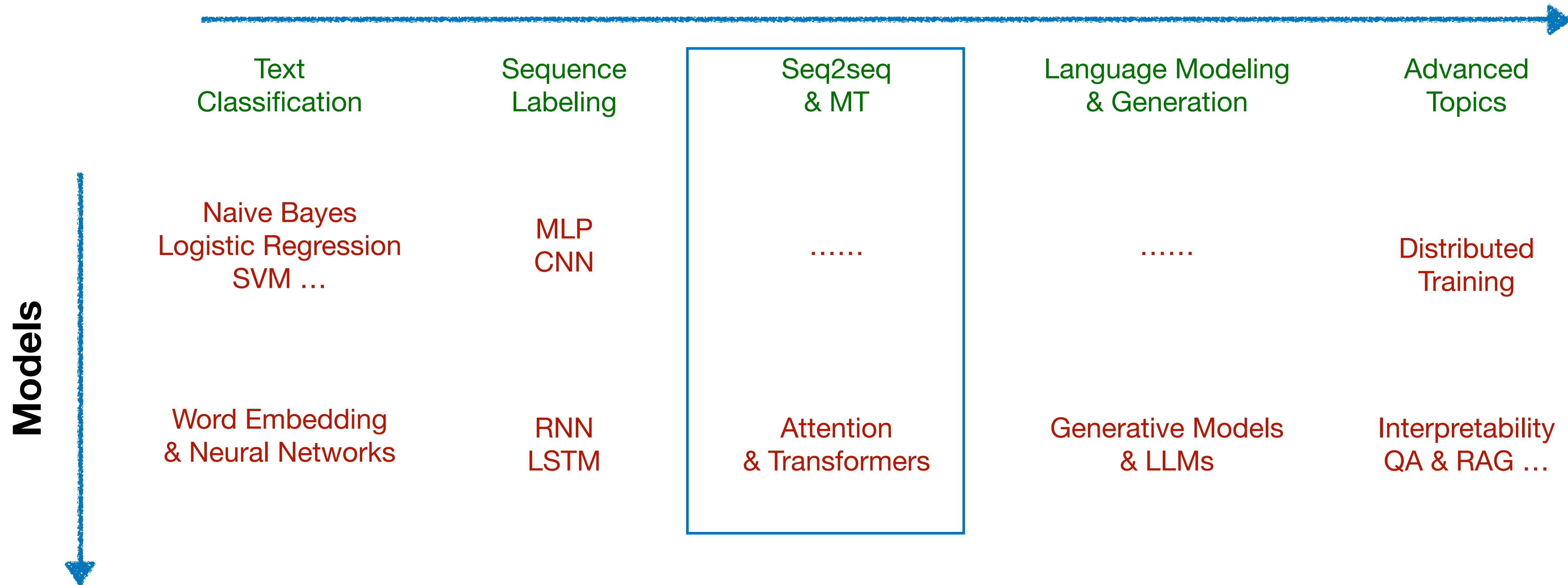
Xuezhe Ma (Max)



USC University of
Southern California

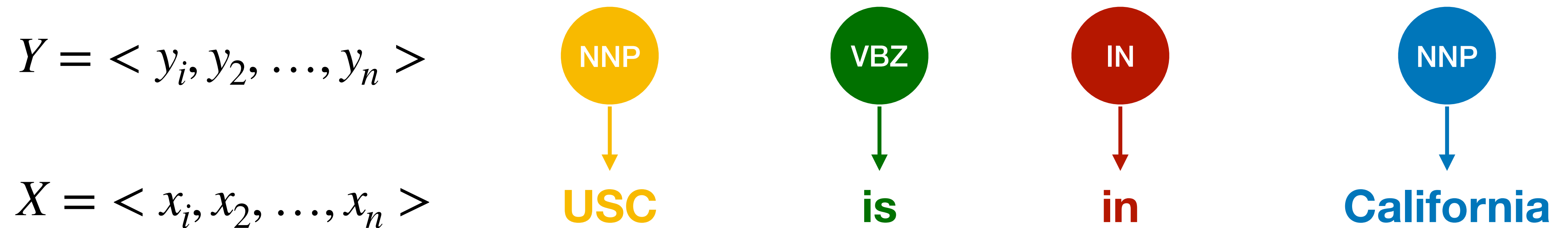
Course Organization

NLP Tasks



Recap: What is Sequence Labeling?

A type of structured prediction tasks

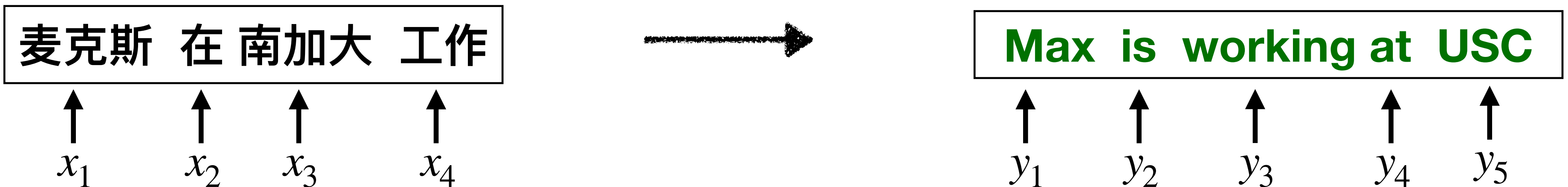


Assigning each token of X , e.g. x_i a corresponding label y_i

Seq2seq Generation

- Sequence-to-Sequence (Seq2seq) Generation

- Input: $X = \{x_1, x_2, \dots, x_L\}, x_i \in \mathcal{X}$
- Output: $Y = \{y_1, y_2, \dots, y_T\}, y_i \in \mathcal{Y}$
- Model: $p_\theta(Y|X)$



Seq2seq Generation

- **Sequence-to-Sequence (Seq2seq) Generation**

- Input: $X = \{x_1, x_2, \dots, x_L\}, x_i \in \mathcal{X}$
- Output: $Y = \{y_1, y_2, \dots, y_T\}, y_i \in \mathcal{Y}$
- Model: $p_{\theta}(Y|X)$

<u>Input X</u>	<u>Output Y (Text)</u>	<u>Task</u>
English	Japanese	Translation
Document	Short Description	Summarization
Utterance	Response	Response Generation
Image	Text	Image Captioning
Speech	Transcript	Speech Recognition

Seq2seq Generation

- **Sequence-to-Sequence (Seq2seq) Generation**
 - Input: $X = \{x_1, x_2, \dots, x_L\}, x_i \in \mathcal{X}$
 - Output: $Y = \{y_1, y_2, \dots, y_T\}, y_i \in \mathcal{Y}$
 - Model: $p_\theta(Y|X)$ **How?**
- **Difference from Sequence Labeling**
 - The length of Y can be different from the length of X
 - The space of $\mathcal{Y}$ is often much larger

麦克斯 在 南加大 工作

$\uparrow$ $\uparrow$ $\uparrow$ $\uparrow$

x_1 x_2 x_3 x_4



Max is working at USC

$\uparrow$ $\uparrow$ $\uparrow$ $\uparrow$ $\uparrow$

y_1 y_2 y_3 y_4 y_5

Statistic Machine Translation

Statistical Machine Translation

- **IBM Translation Models**
 - Word-level alignment model
 - EM algorithm
- **Phrase-based Translation Models**
 - Phrase-based alignment model
- **Heavy Engineering**
 - Moses system
 - 360 pages manual

Statistical Machine Translation

- **IBM Translation Models**
 - Word-level alignment model
 - EM algorithm
- **Phrase-based Translation Models**
 - Phrase-based alignment model
- **Heavy Engineering**
 - Moses system
 - 360 pages manual

Word-Alignment Model in SMT

- **Key Idea:** two words are more likely to be aligned when they occur more frequently in translation pairs

我 不 知道

I don't know

我 是 学生

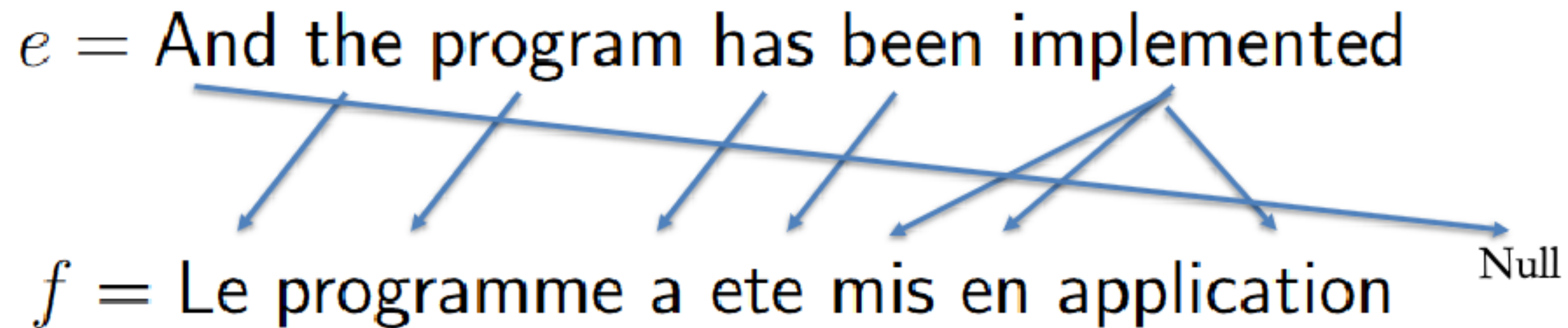
I am a student

我 爱 喝 茶

I love drinking tea

Word-Alignment Model in SMT

- e is an English sentence with l words
- f is a foreign sentence with m words
- An alignment $a = \{a_1, a_2, \dots, a_m\}$, $a_j \in \{0, \dots, l\}$
- Hence there are $(l + 1)^m$ possible alignments



Word-Alignment Model in SMT

- IBM Model 1:

$$p(a | e, m) = \frac{1}{(l + 1)^m}$$

- IBM Model 2:

$$p(a | e, m) = \prod_{j=1}^m q(a_j | j, l, m)$$

- IBM Model 3, 4, 5, 6...

Translation Model

$$p(f | e) = \sum_{a \in \mathcal{A}} p(a | e, m) p(f | a, e, m)$$

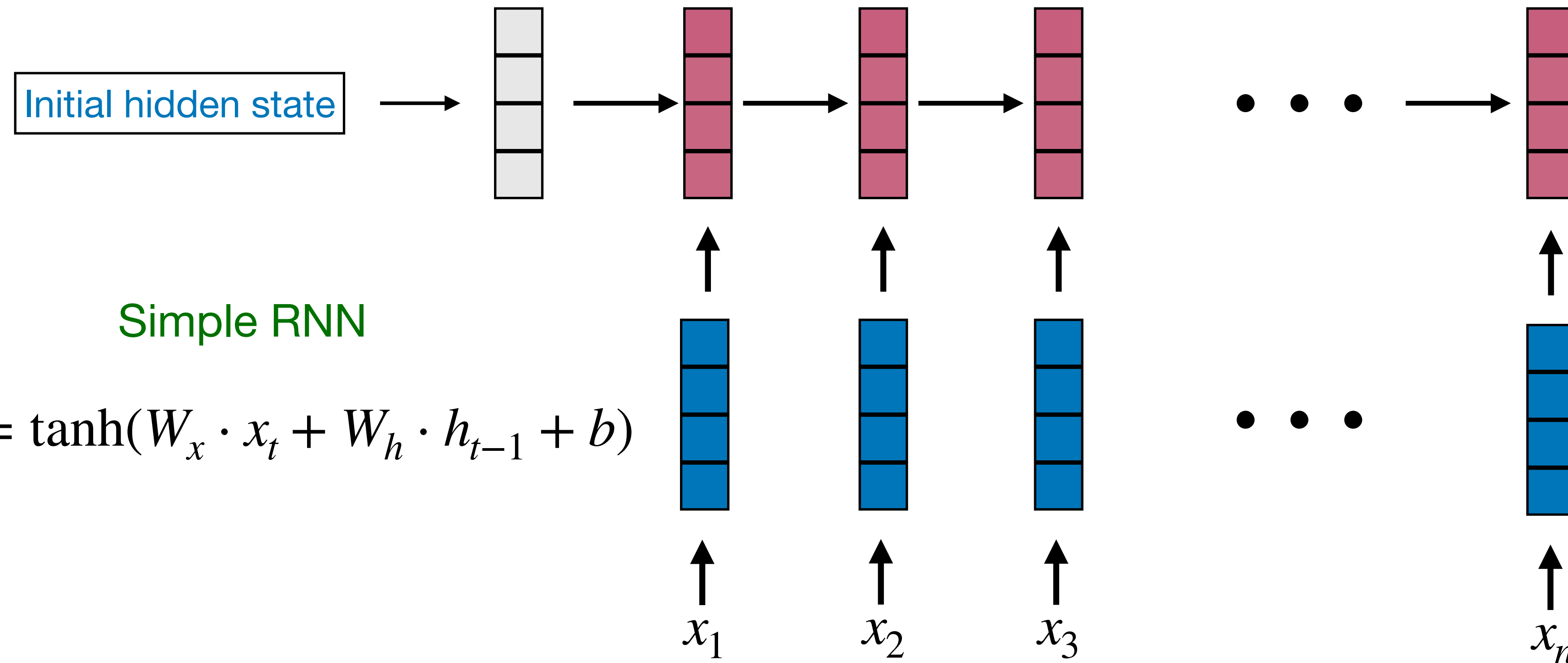
Statistical Machine Translation

- **IBM Translation Models**
 - Word-level alignment model
 - EM algorithm
- **Phrase-based Translation Models**
 - Phrase-based alignment model
- **Heavy Engineering**
 - Moses system
 - 360 pages manual

Neural Machine Translation

Recap: RNN & LSTM

- **Recurrent Neural Networks (RNNs)**
 - Re-using one feed forward network in a recurrent way



Recap: RNN & LSTM

$$\mathbf{i}_t = \sigma(\mathbf{W}^i \mathbf{h}_{t-1} + \mathbf{U}^i \mathbf{x}_t + \mathbf{b}^i)$$

Input gate: $i_t \in (0,1)$

$$\mathbf{f}_t = \sigma(\mathbf{W}^f \mathbf{h}_{t-1} + \mathbf{U}^f \mathbf{x}_t + \mathbf{b}^f)$$

Forget gate: $f_t \in (0,1)$

$$\mathbf{o}_t = \sigma(\mathbf{W}^o \mathbf{h}_{t-1} + \mathbf{U}^o \mathbf{x}_t + \mathbf{b}^o)$$

Output gate: $o_t \in (0,1)$

$$\mathbf{g}_t = \tanh(\mathbf{W}^g \mathbf{h}_{t-1} + \mathbf{U}^g \mathbf{x}_t + \mathbf{b}^g)$$

Simple RNN

$$\mathbf{c}_t = \mathbf{c}_{t-1} \odot \mathbf{f}_t + \mathbf{g}_t \odot \mathbf{i}_t$$

Cell state vector (internal memory)

$$\mathbf{h}_t = \tanh(\mathbf{c}_t) \odot \mathbf{o}_t$$

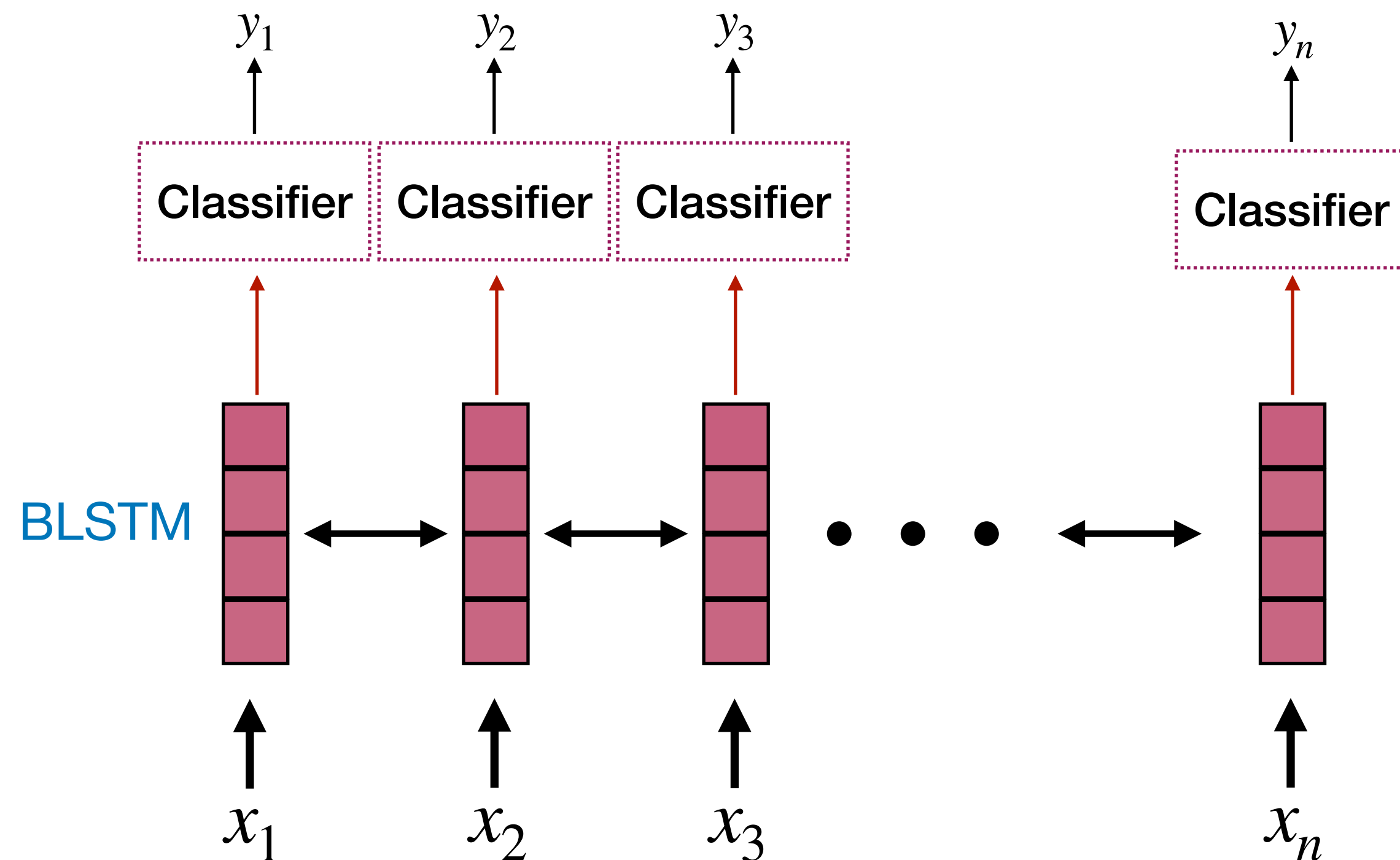
Final hidden vector

Autoregressive Seq2seq Generation

- Sequence labeling vs. Seq2seq Generation

Sequence labeling

$$p_{\theta}(Y|X) = \prod_{t=1}^T p_{\theta}(y_t|X)$$



Why not for seq2seq generation?

Autoregressive Seq2seq Generation

$$p_{\theta}(Y|X) = \prod_{t=1}^T p_{\theta}(y_t|X)$$

Not a good choice!

我 不 知道

I don't know

I do not know

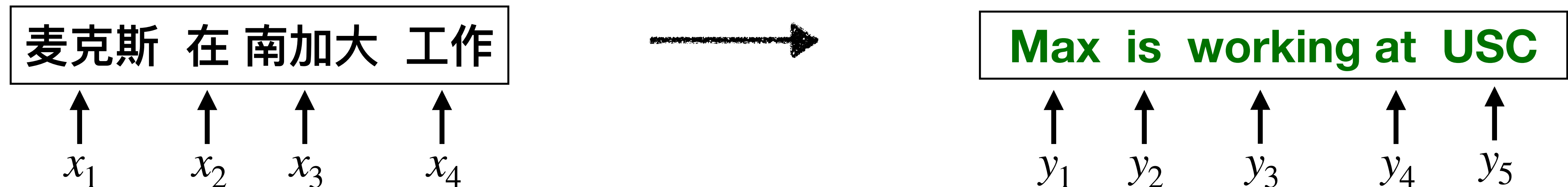
I have no idea

Autoregressive Seq2seq Generation

- Autoregressive Factorization:

$$p_{\theta}(Y|X) = \prod_{t=1}^T p_{\theta}(y_t | y_{<t}, X)$$


- Autoregressive factorization is just chain-rule
- Autoregressive factorization does **NOT** assume any independence
- With autoregressive factorization, we need to model each $p_{\theta}(y_t | y_{<t}, X)$

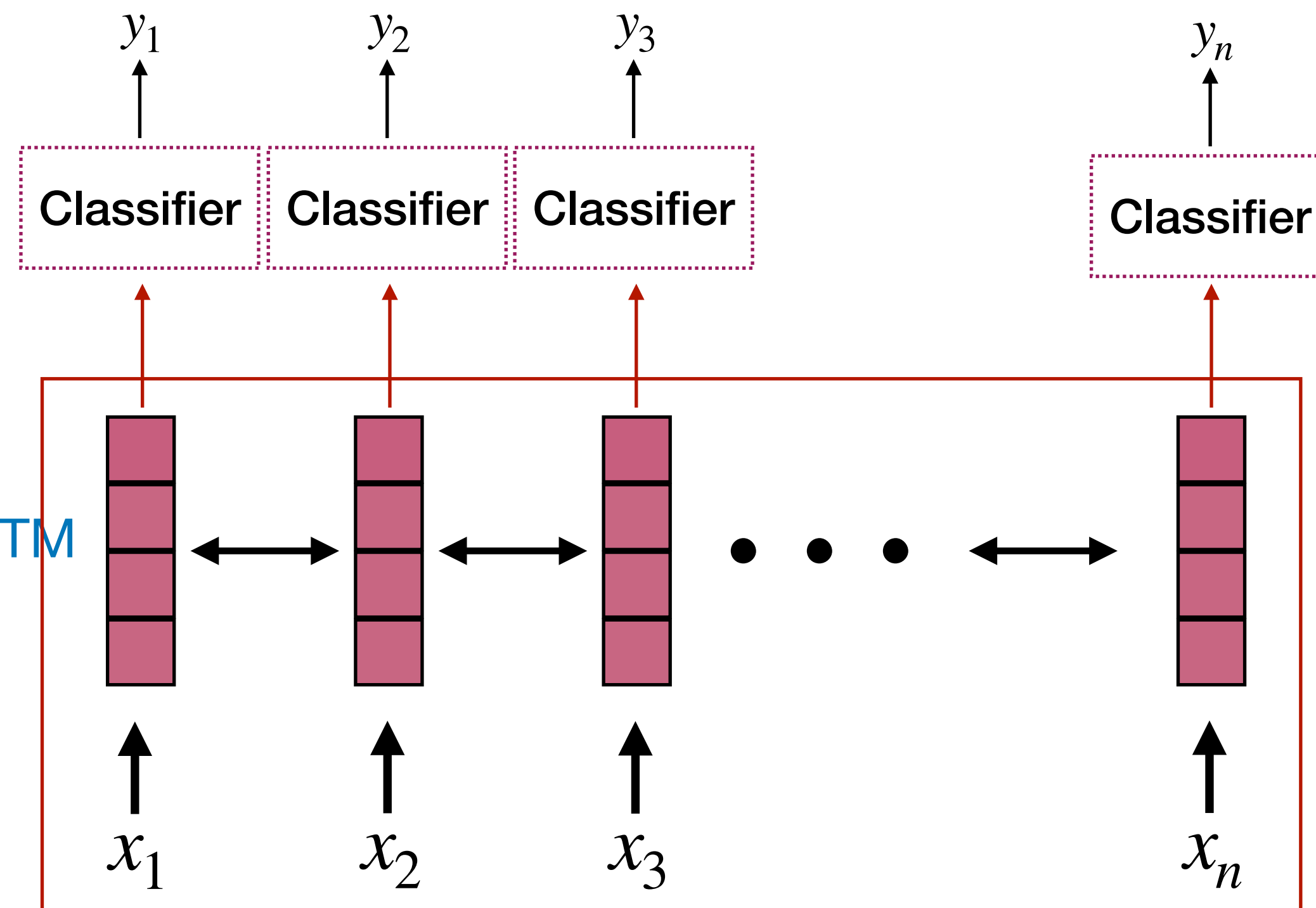


Encoder-Decoder Architecture

- Sequence labeling vs. Seq2seq Generation

Sequence labeling

$$p_{\theta}(Y|X) = \prod_{t=1}^T p_{\theta}(y_t | \boxed{X})$$



Seq2seq Generation

$$p_{\theta}(Y|X) = \prod_{t=1}^T p_{\theta}(\boxed{y_t} | y_{<t}, \boxed{X})$$

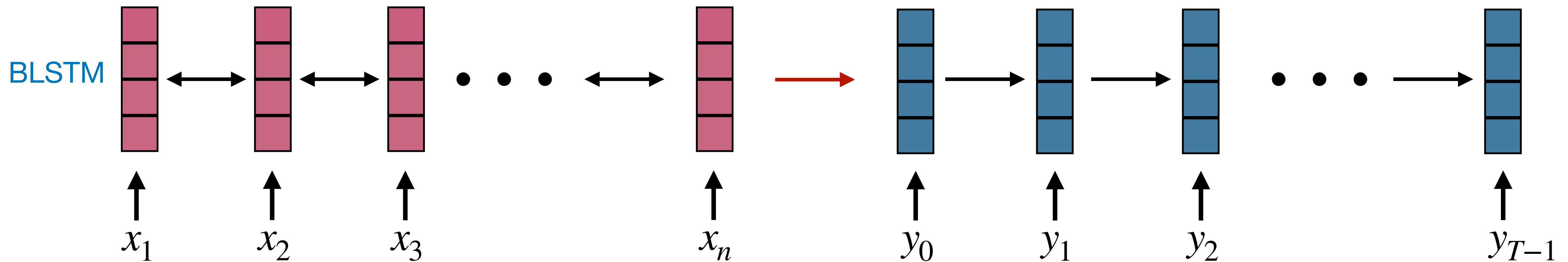
Encoder: encode a sentence into a sequence of vectors

Decoder: use another LSTM?

Encoder-Decoder Architecture

- **Two Components:**
 - **Encoder:** Convert input sequence into a sequence of vectors
 - **Decoder:** Convert encoded vectors into a sequence in the output space

$$p_{\theta}(Y|X) = \prod_{t=1}^T p_{\theta}(\boxed{y_t | y_{<t}}, X)$$



Special Tokens in Seq2seq

- **<BOS>**: start of the target sentence
- **<EOS>**: end of the target sentence

麦克斯 在 南加大 工作

x_1 x_2 x_3 x_4

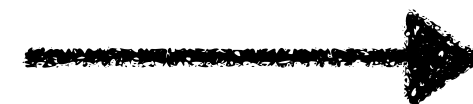


Max is working at USC

y_1 y_2 y_3 y_4 y_5

麦克斯 在 南加大 工作

x_1 x_2 x_3 x_4



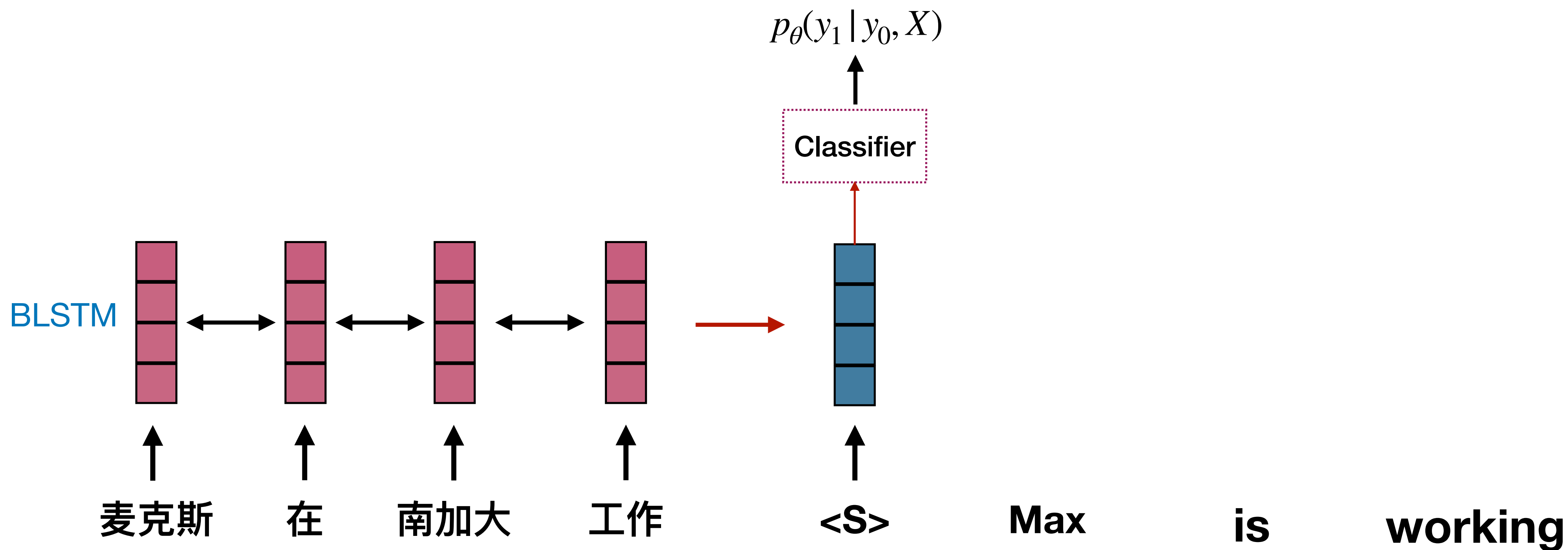
<BOS> Max is working at USC **<EOS>**

y_0 y_1 y_2 y_3 y_4 y_5 y_6

Seq2seq Training

- Model Training:

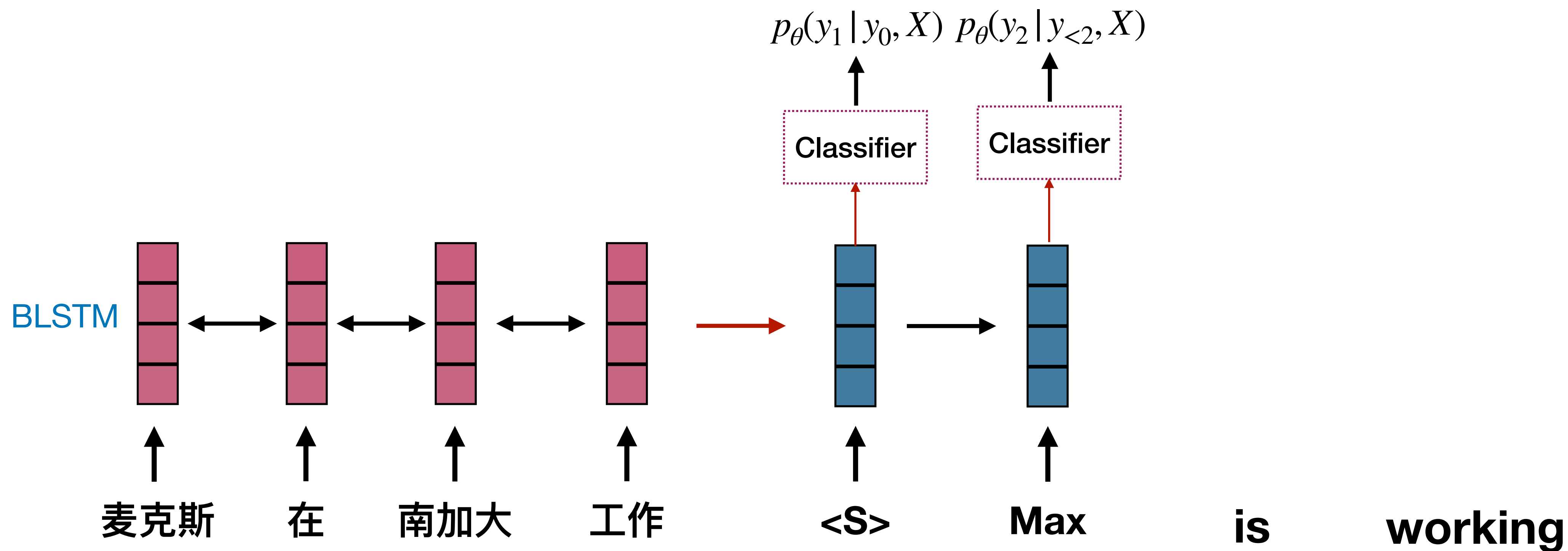
$$p_{\theta}(Y|X) = \prod_{t=1}^T p_{\theta}(y_t | y_{<t}, X) \quad t = 1$$



Seq2seq Training

- Model Training:

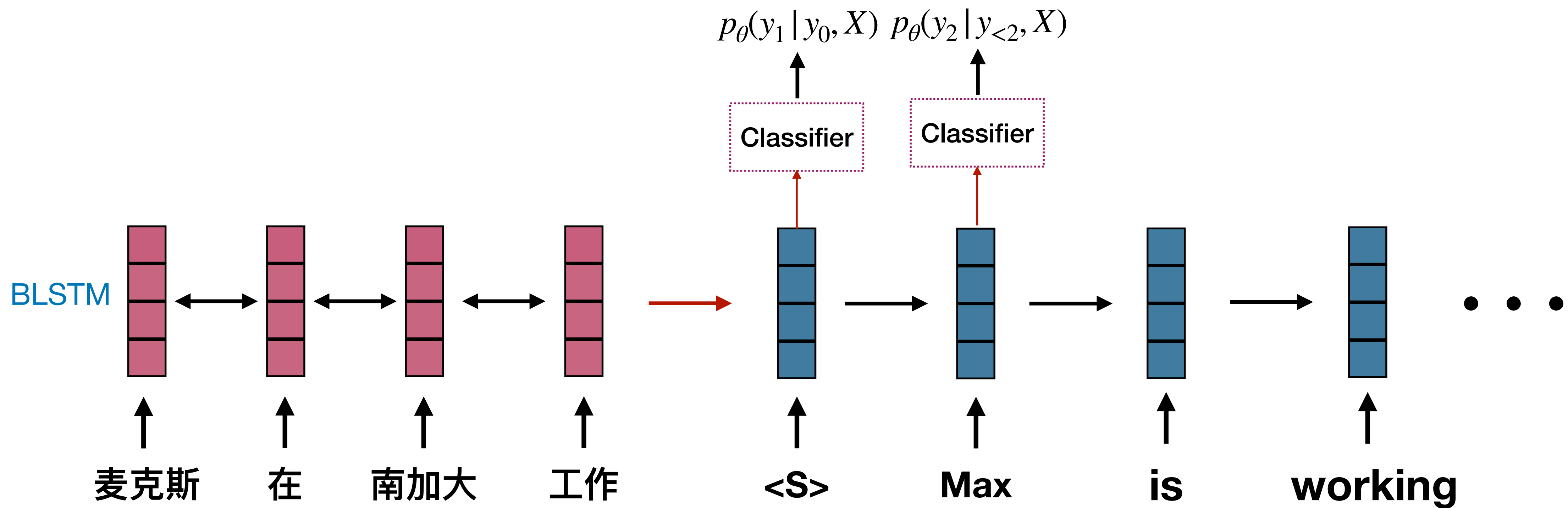
$$p_{\theta}(Y|X) = \prod_{t=1}^T p_{\theta}(y_t | y_{<t}, X) \quad t = 2$$



Seq2seq Training

- Model Training:

$$p_{\theta}(Y|X) = \prod_{t=1}^T p_{\theta}(y_t | y_{<t}, X)$$



Seq2seq Training

- Maximum Likelihood Estimation

$$\max_{\theta} p_{\theta}(Y|X) = \prod_{t=1}^T p_{\theta}(y_t | y_{<t}, X)$$

- Back-propagate gradients through both decoder & encoder
- Need a really big training corpus
 - WMT Russian-English



Seq2seq Decoding

- Exhaustive Search
 - Requires computing all possible sequences

$$\arg \max_Y p_{\theta}(Y|X) = \prod_{t=1}^T p_{\theta}(y_t | y_{<t}, X)$$

What is the complexity of doing this search, if $|\mathcal{Y}| = V$ and sequence length T ?

- (a) $O(VT)$
- (b) $O(V^T)$
- (c) $O(T^V)$

Seq2seq Decoding

- Greedy Search

- Selects the best current word y_t

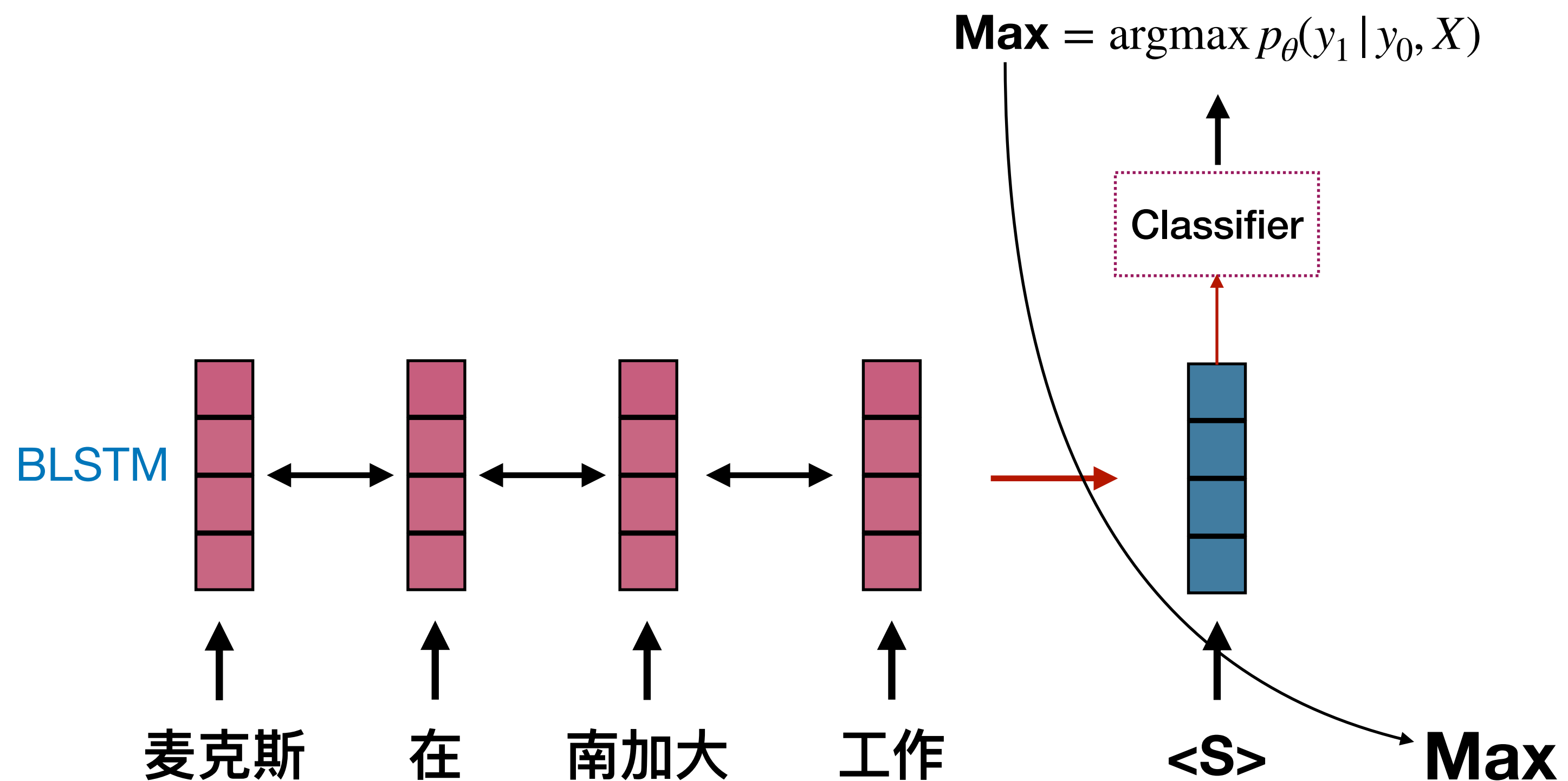
$$\arg \max_Y p_{\theta}(Y|X) = \prod_{t=1}^T p_{\theta}(y_t | y_{<t}, X)$$

$$\approx \mathbf{arg} \max_{y_t} p_{\theta}(y_t | y_{<t}, X), \forall t$$

Seq2seq Decoding

- Greedy decoding:

$$y_t^* = \arg \max_{y_t} p_{\theta}(y_t | y_{<t}, X), \forall t$$

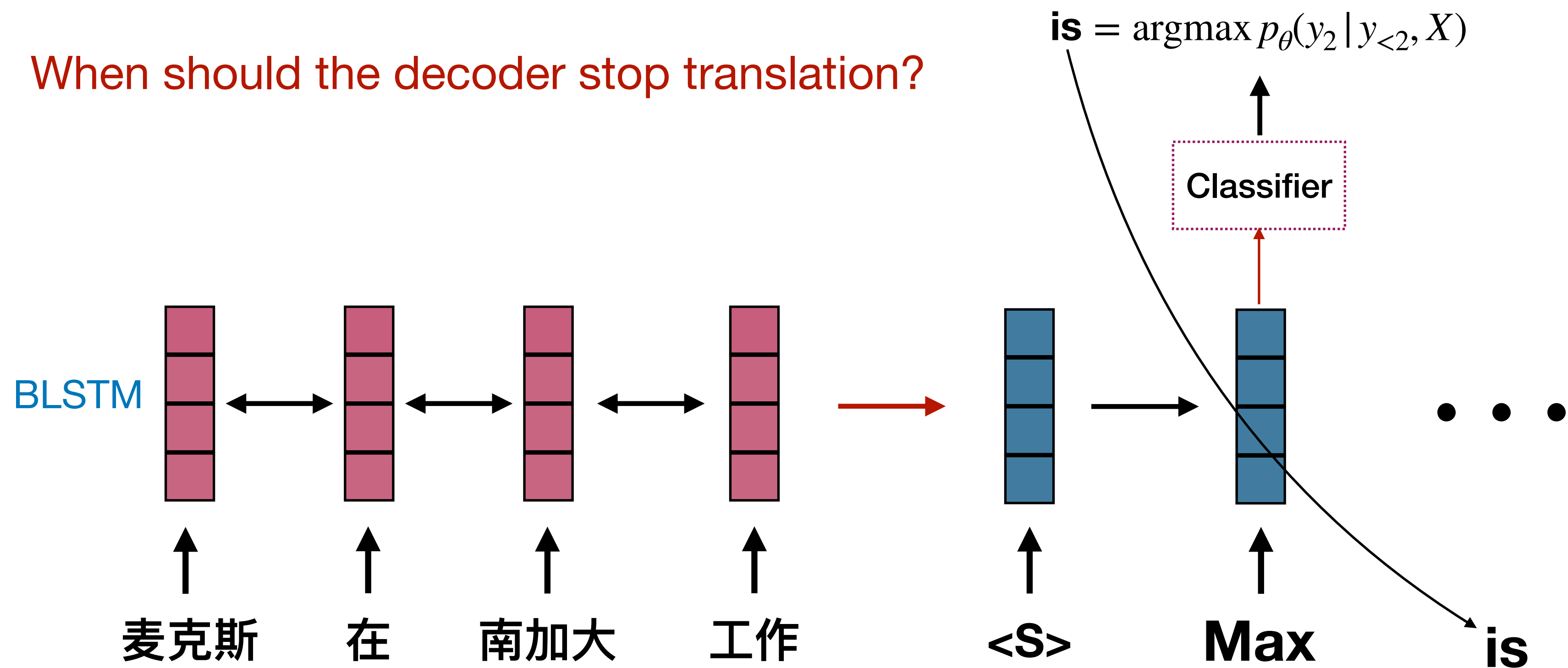


Seq2seq Decoding

- Greedy decoding:

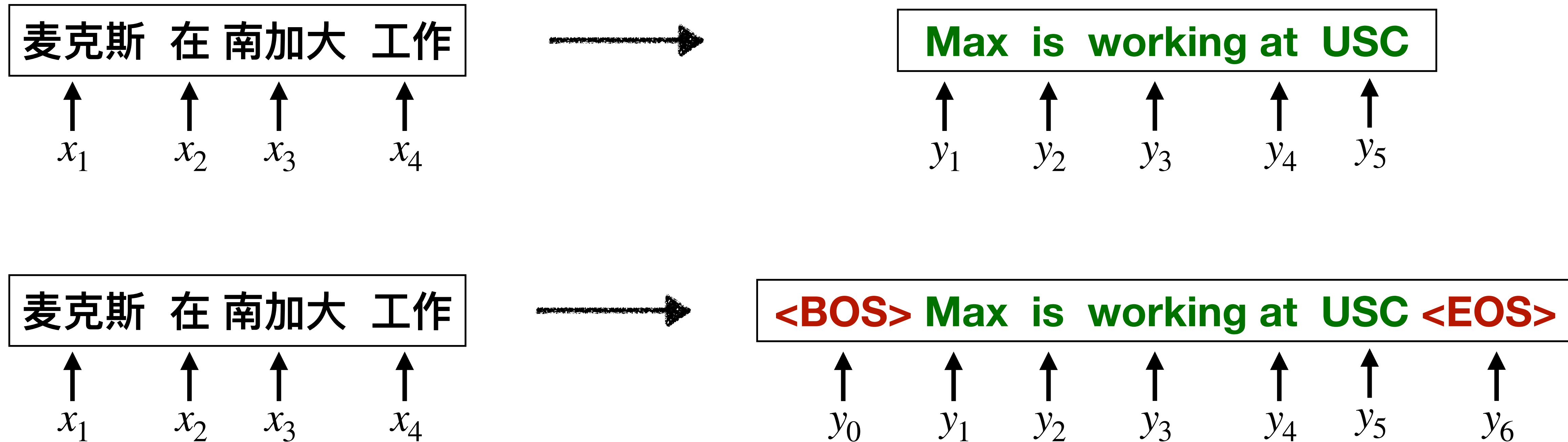
$$y_t^* = \arg \max_{y_t} p_{\theta}(y_t | y_{<t}, X), \forall t$$

When should the decoder stop translation?



Special Tokens in Seq2seq

- **<BOS>**: start of the target sentence
- **<EOS>**: end of the target sentence

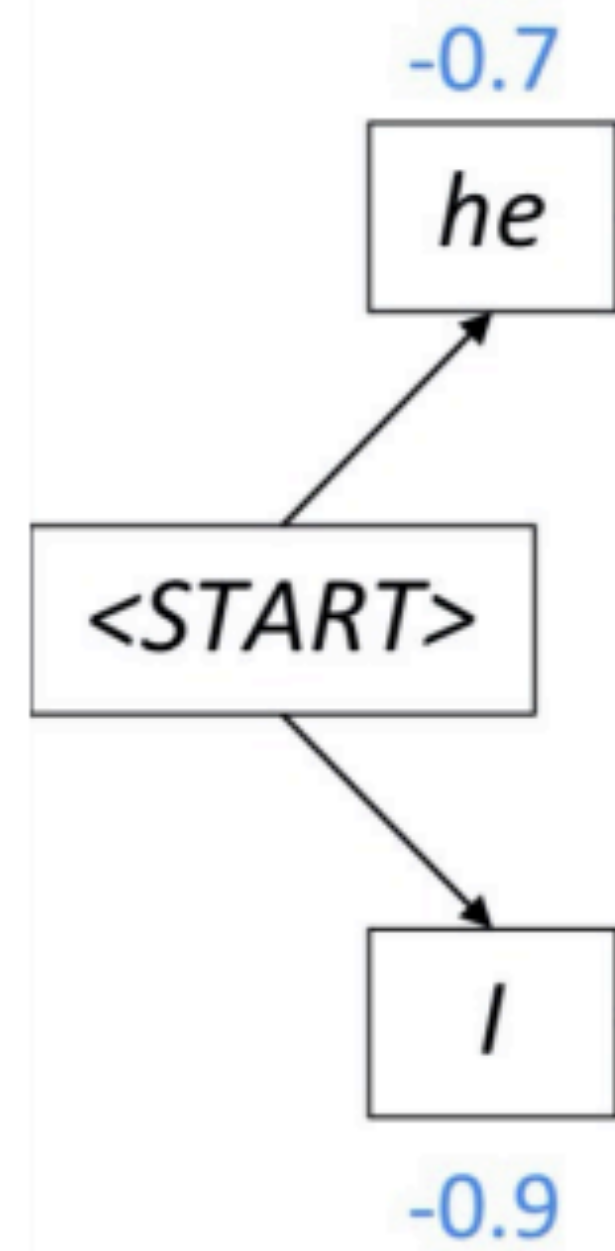


A Middle Ground: Beam Search

- **Key idea:** at every step, keep track of the **k most probable** partial translations (hypotheses)
- Score of each hypothesis = log probability of sequence so far
- Not guaranteed to be optimal
- More efficient than exhaustive search

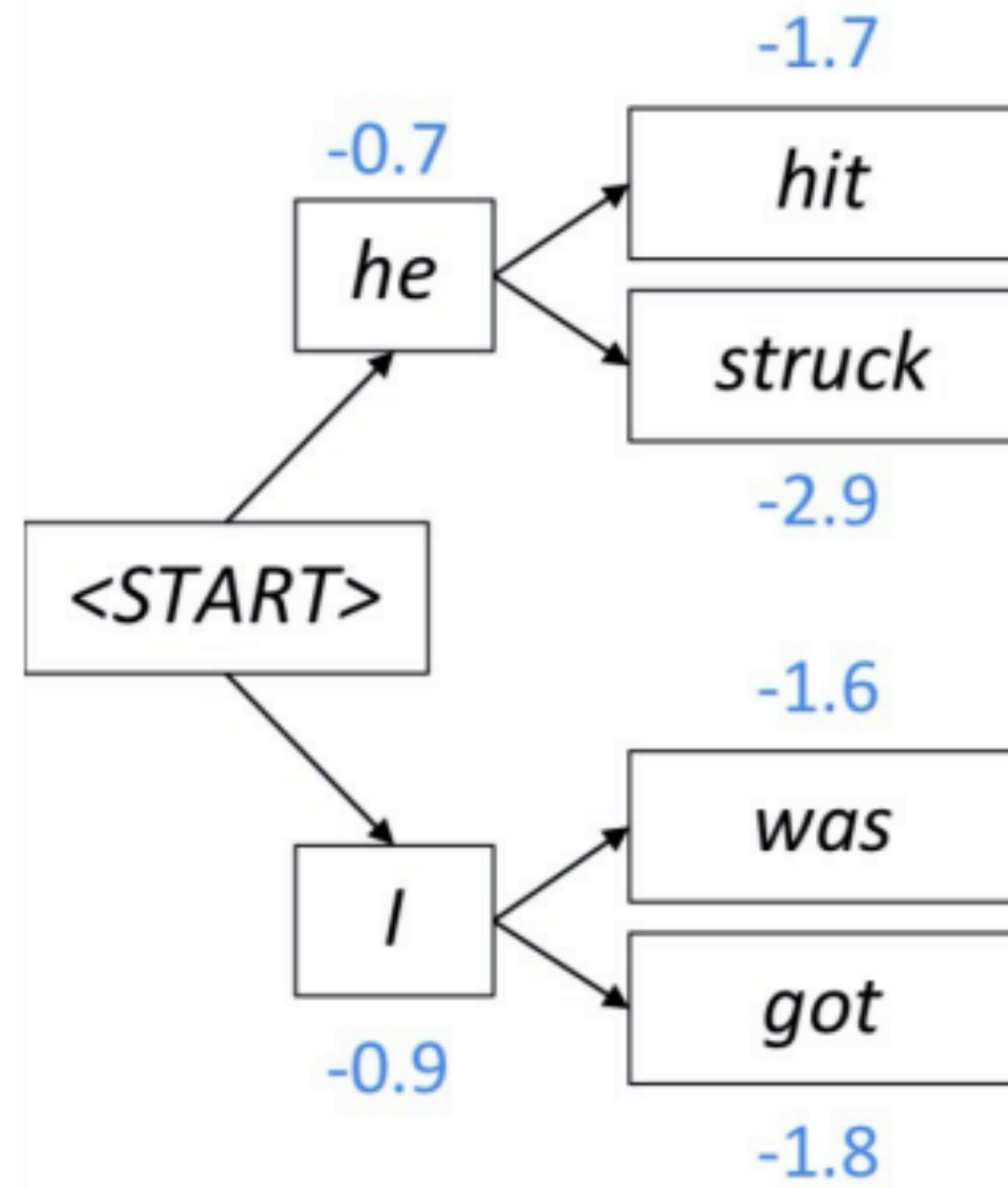
Beam Search Decoding

Beam size $K = 2$



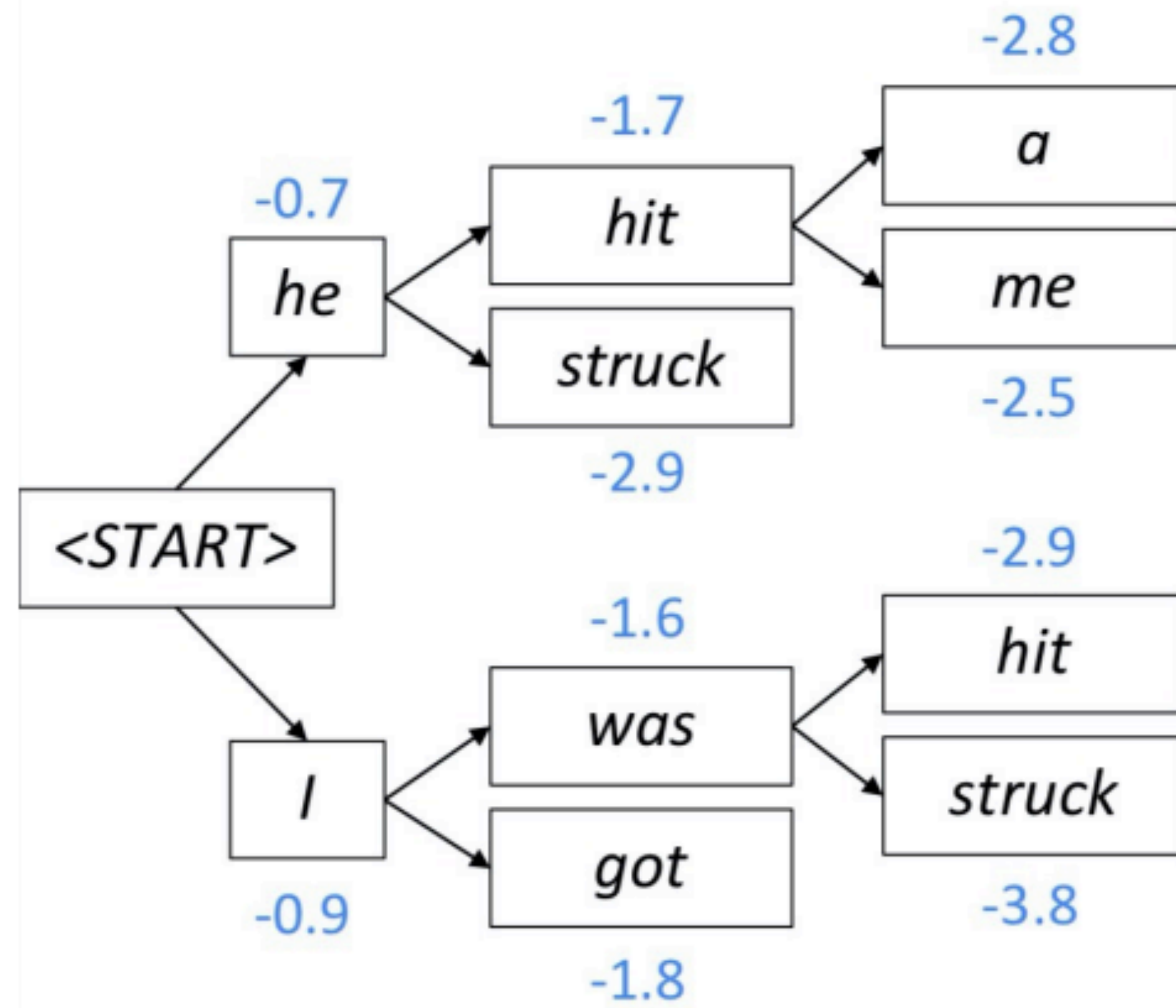
Beam Search Decoding

Beam size $K = 2$



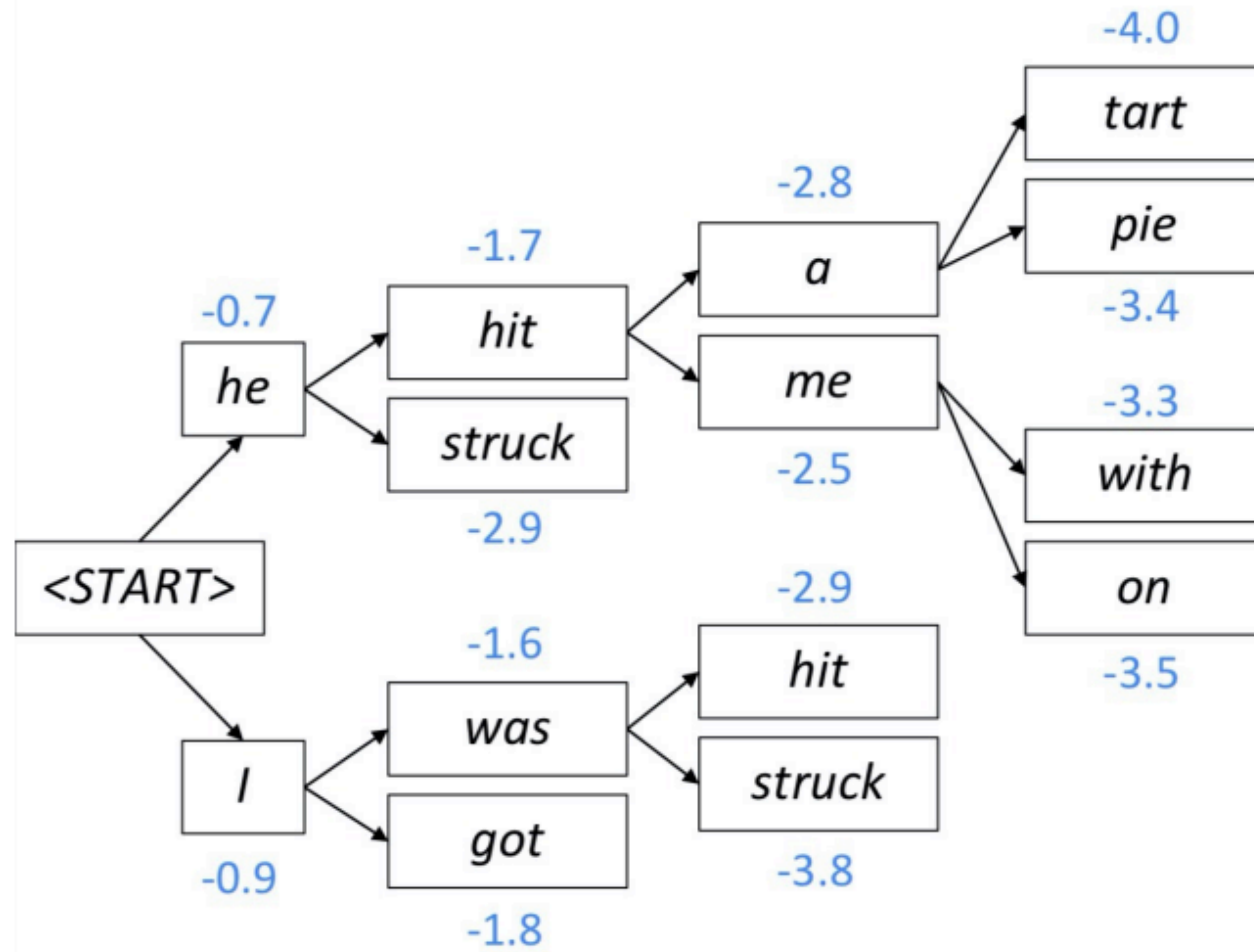
Beam Search Decoding

Beam size $K = 2$



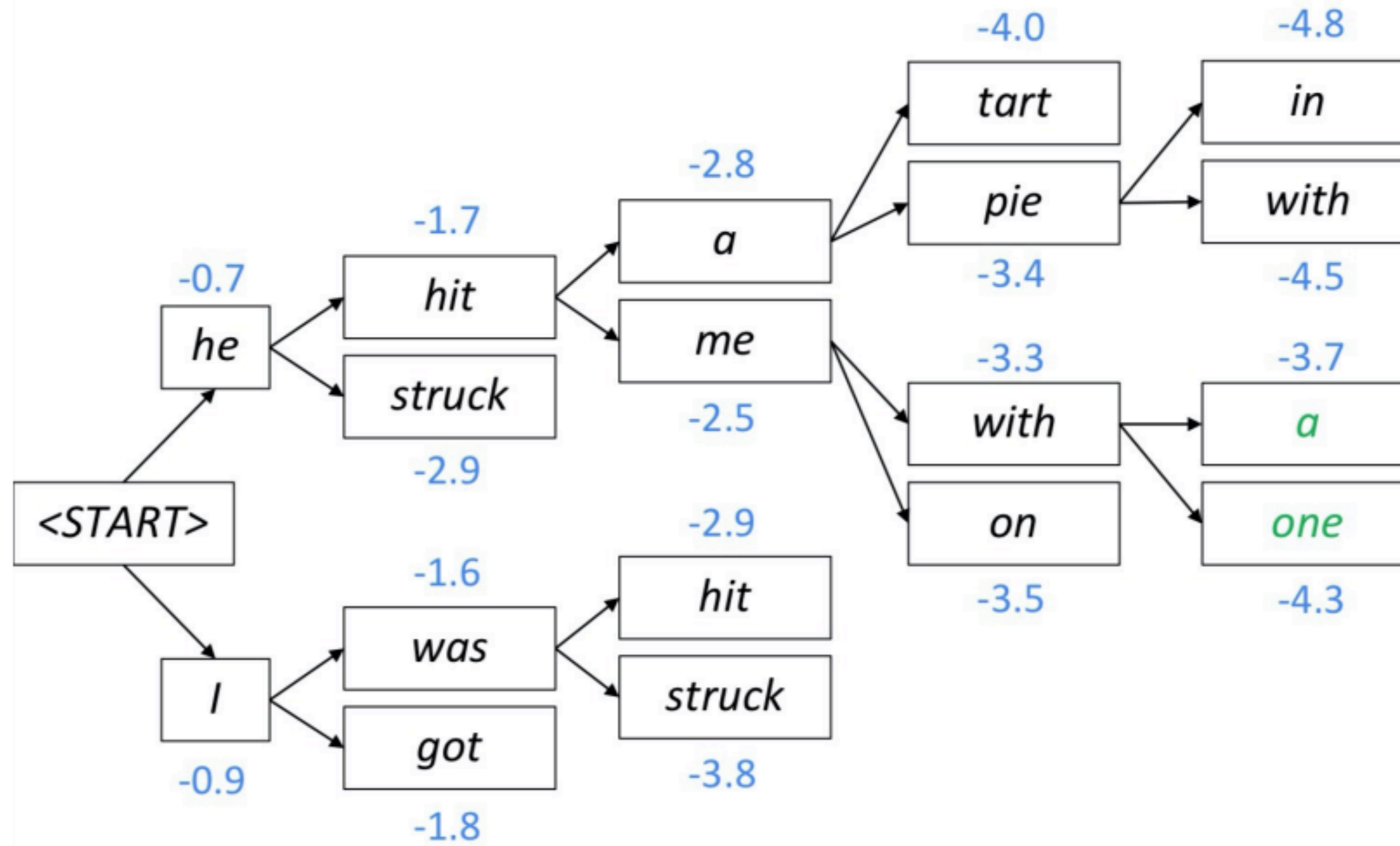
Beam Search Decoding

Beam size $K = 2$



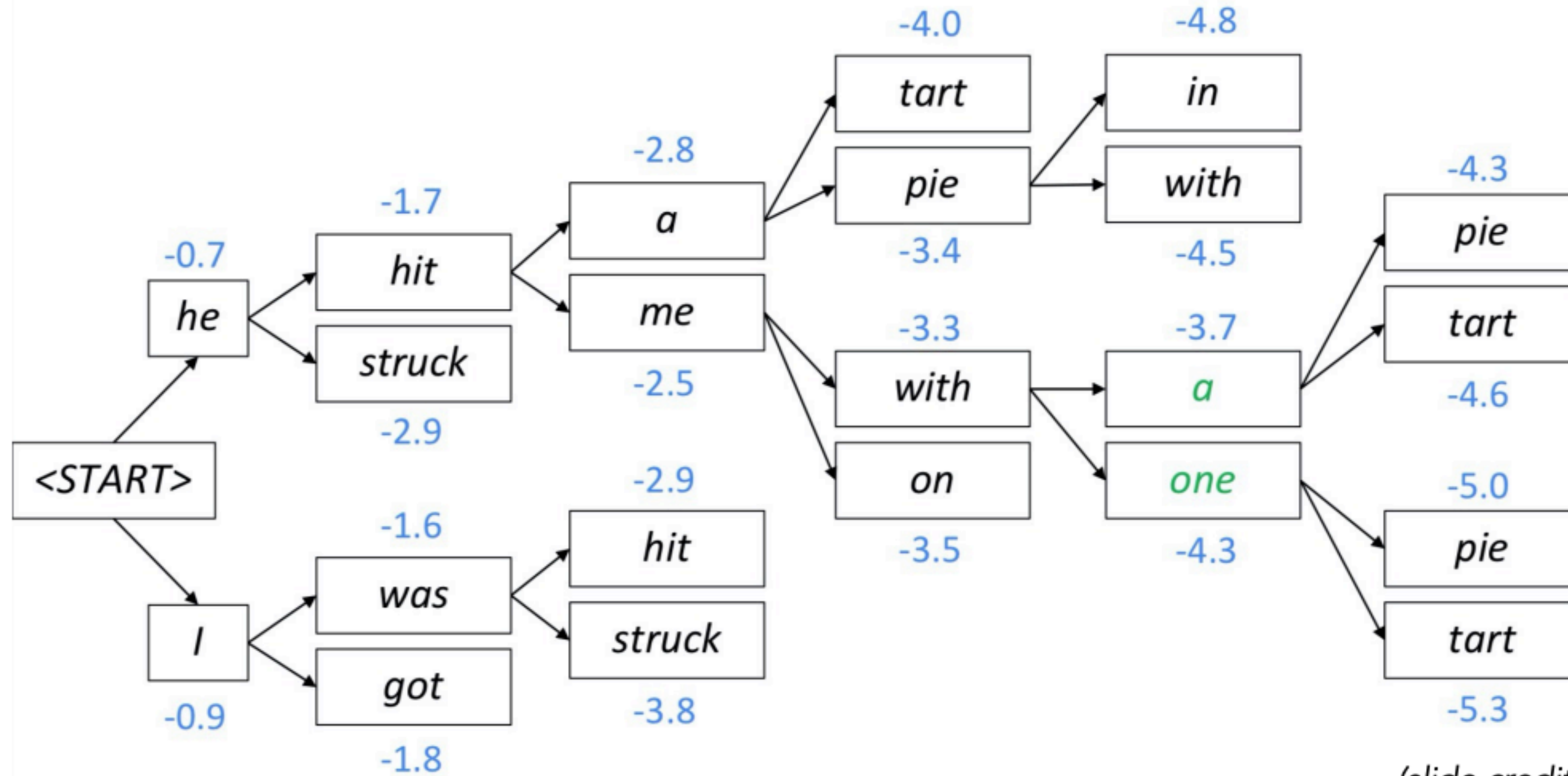
Beam Search Decoding

Beam size $K = 2$



Beam Search Decoding

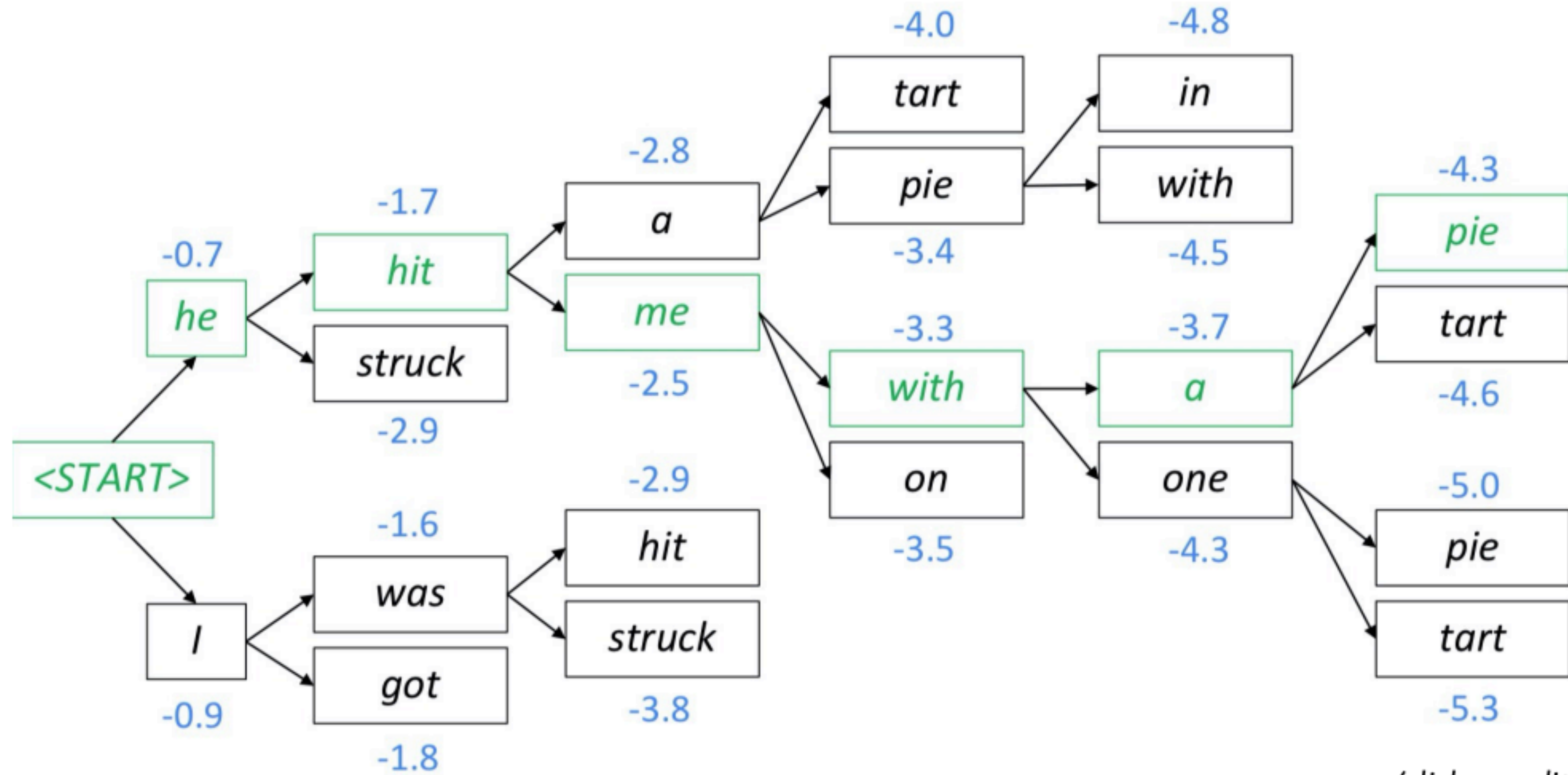
Beam size $K = 2$



(slide credit: Abigail See)

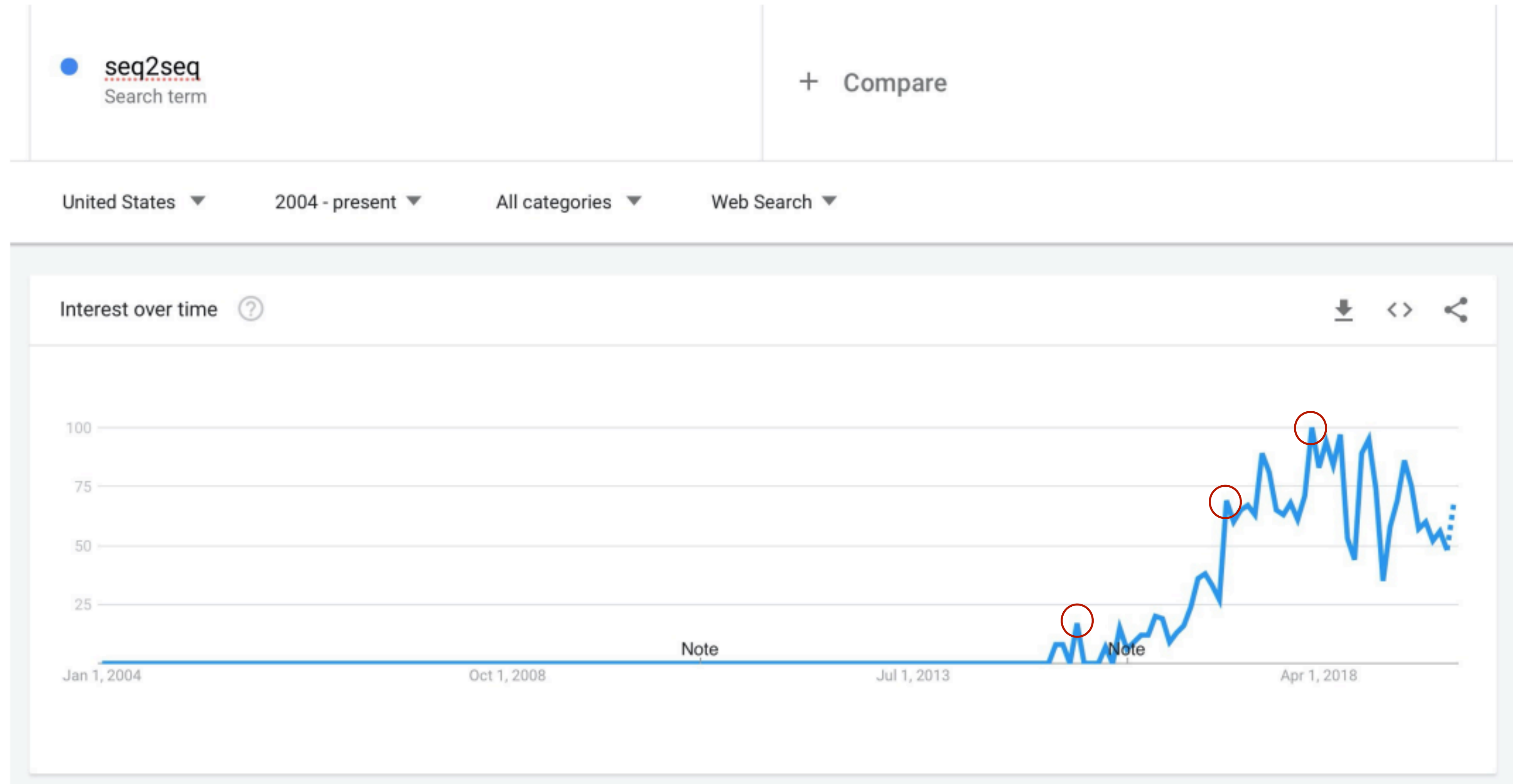
Backtrack

Beam size $K = 2$

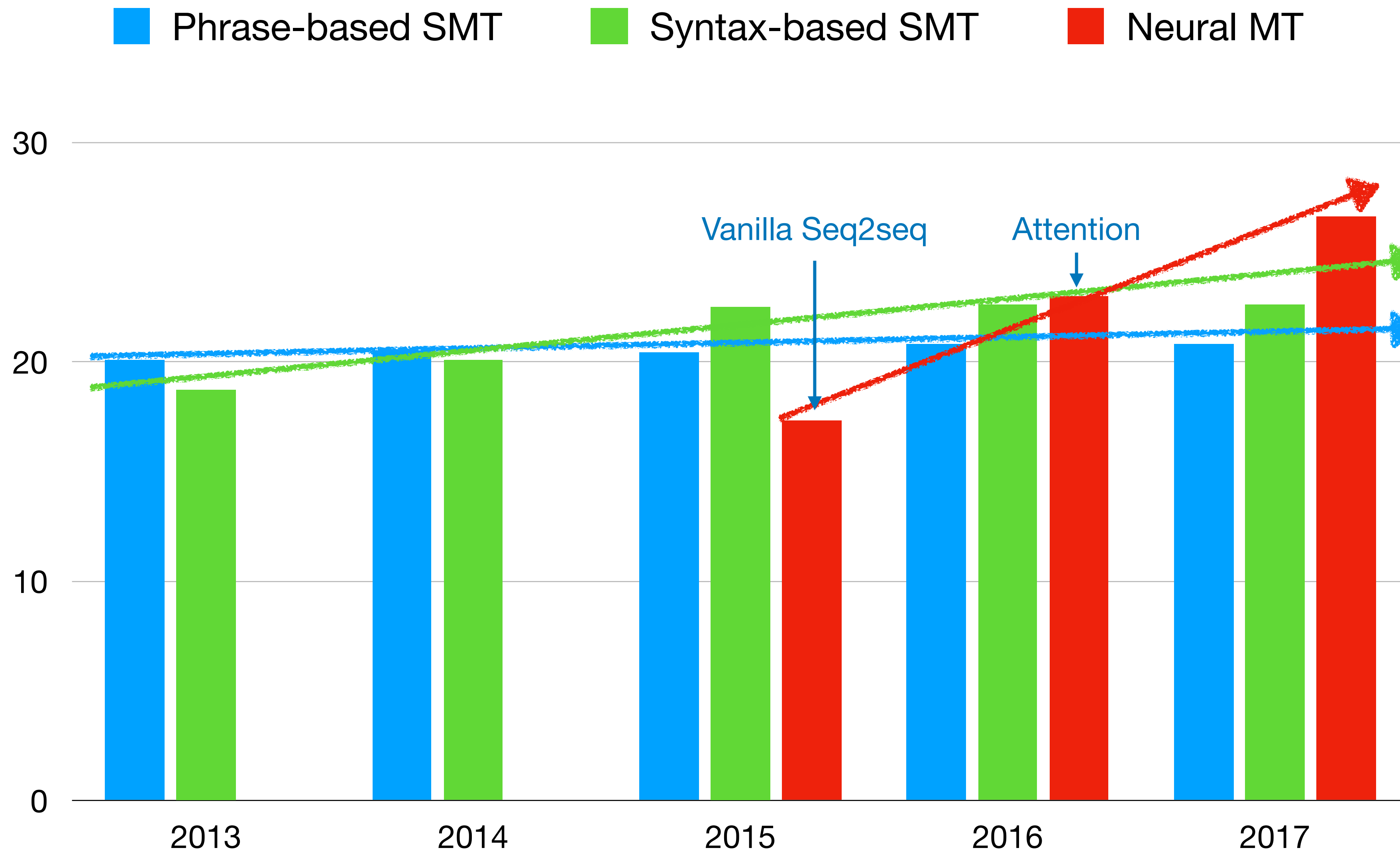


(slide credit: Abigail See)

How Seq2seq changed the MT Landscape

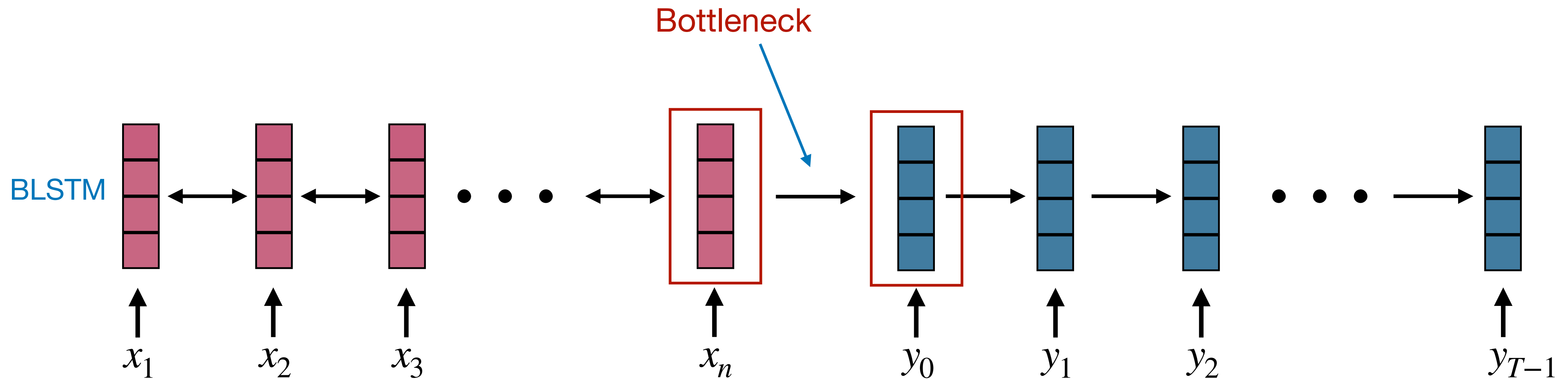


MT Progress



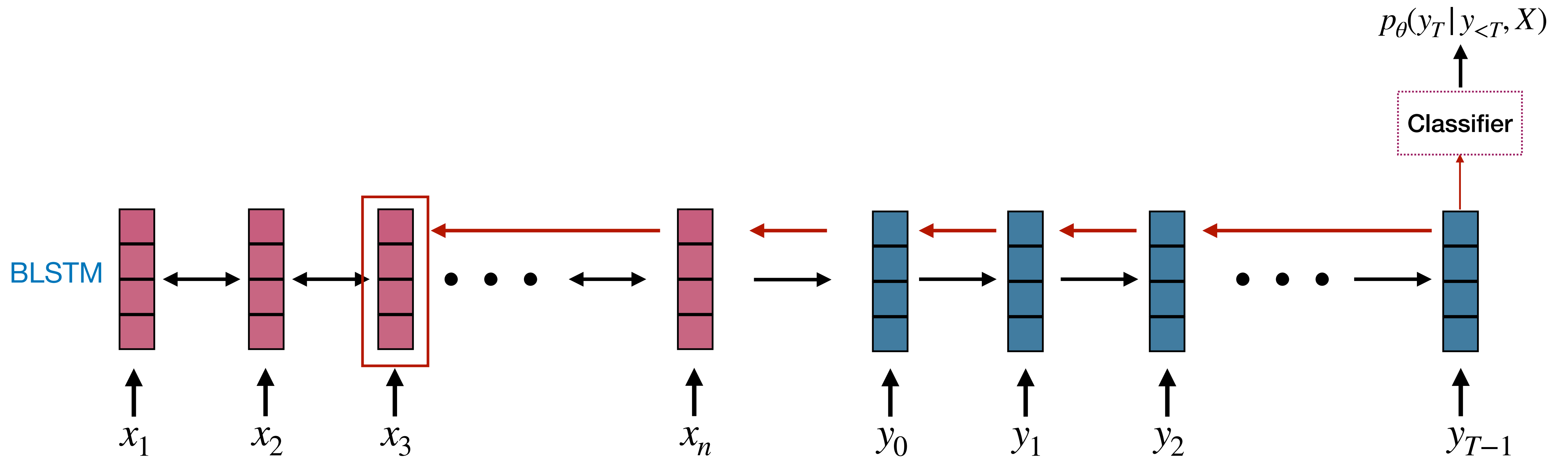
Issues with Vanilla Encoder-Decoder Architecture

- A single encoding vector needs to capture **all the information** about source sentence



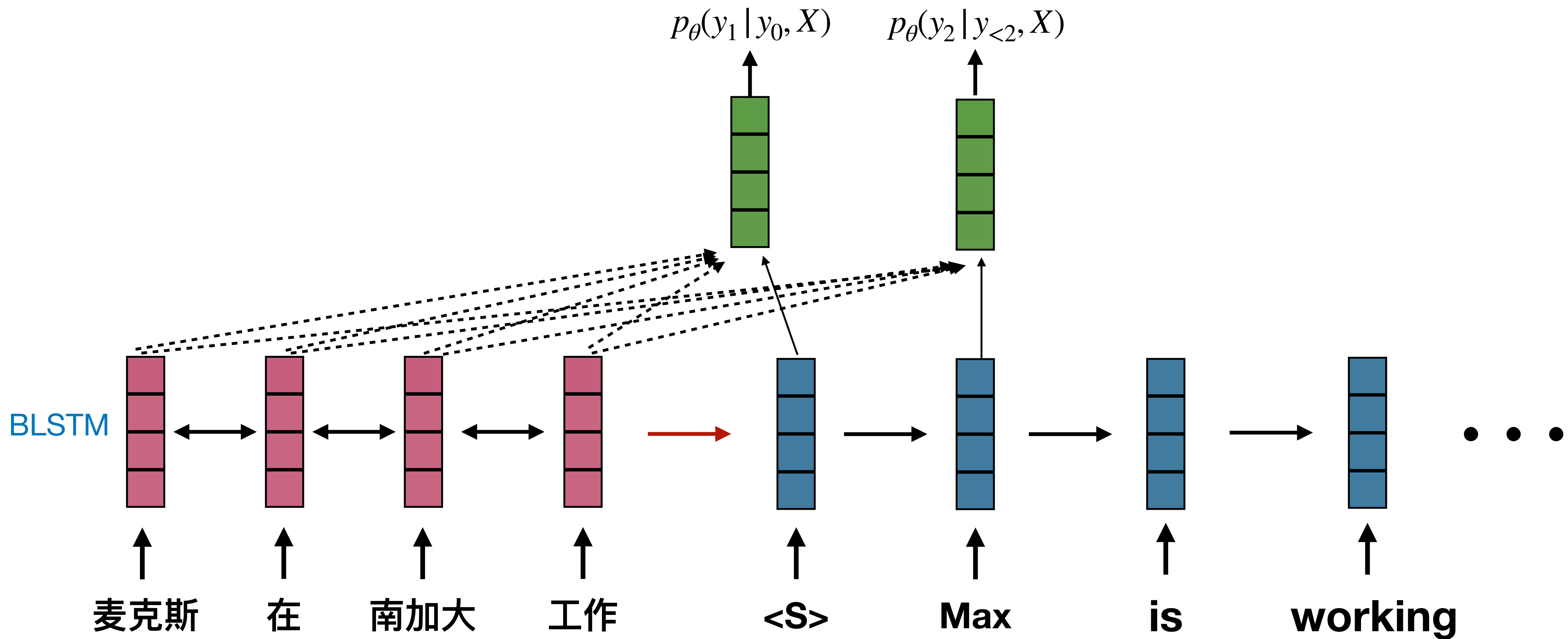
Issues with Vanilla Encoder-Decoder Architecture

- A single encoding vector needs to capture **all the information** about source sentence
- Longer sequences can lead to **vanishing gradients**



Attention Mechanism

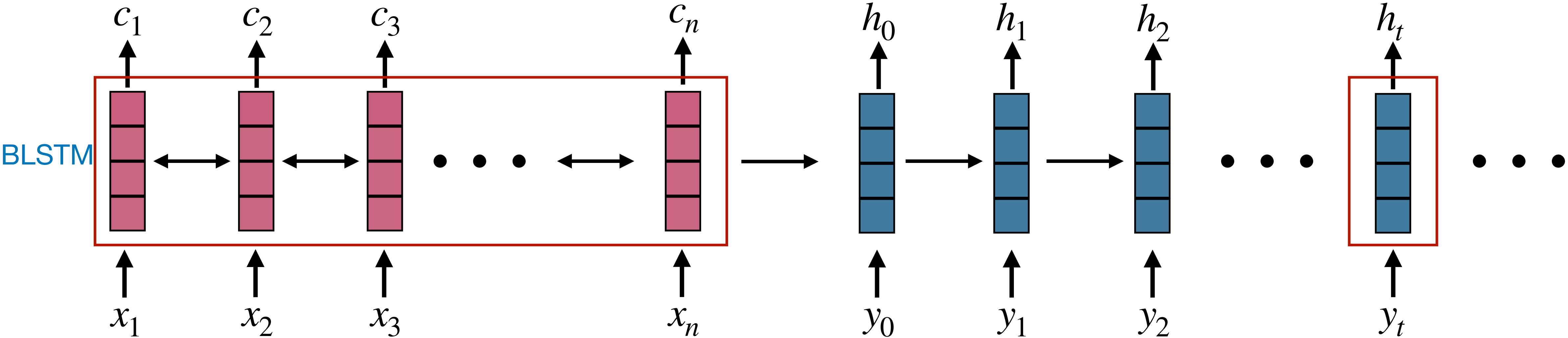
- **Key idea:** At each time step, use all parts of source sentence



Attention Mechanism



$$= \text{attn}([c_1, c_2, \dots, c_n], h_t)$$



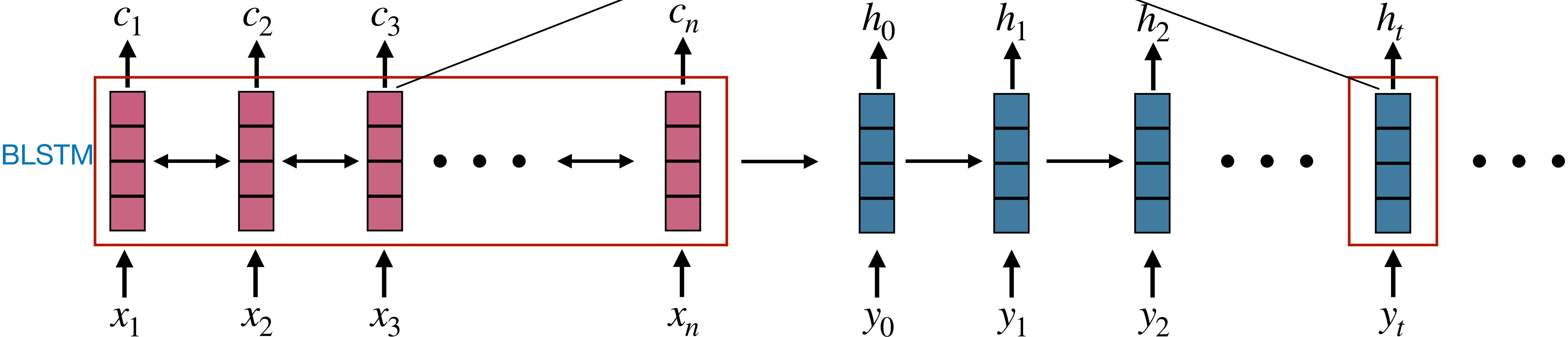
Attention Mechanism



$$= \text{attn}([c_1, c_2, \dots, c_n], h_t)$$

$$e_j^t = \text{sim}(c_j, h_t), \forall j \in \{1, \dots, n\}$$

Attention scores



Attention Mechanism



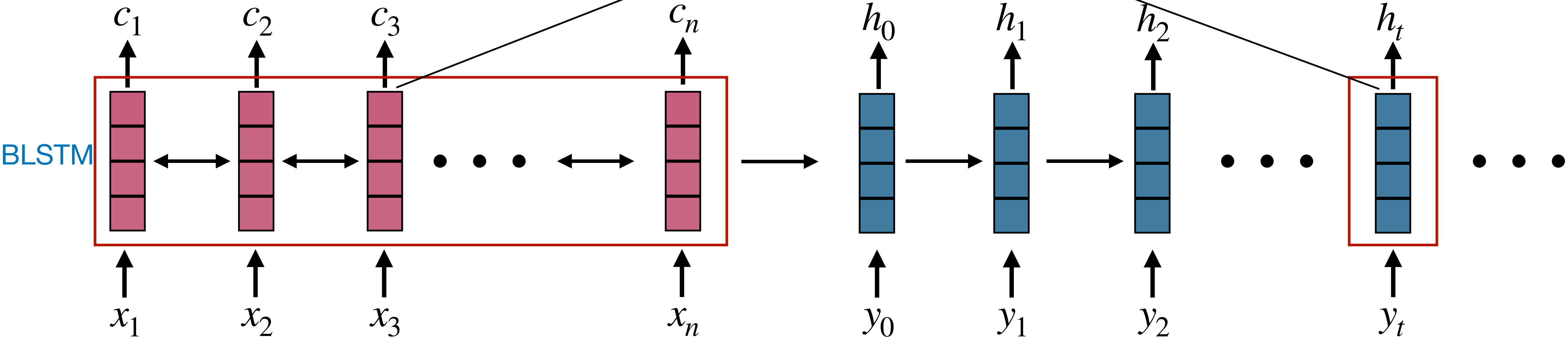
$$= \text{attn}([c_1, c_2, \dots, c_n], h_t)$$

$$a^t = \text{softmax}(e^t) \in (0,1)^n$$

Attention distribution

$$e_j^t = \text{sim}(c_j, h_t), \forall j \in \{1, \dots, n\}$$

Attention scores



Attention Mechanism



$$= \text{attn}([c_1, c_2, \dots, c_n], h_t) = \sum_{j=1}^n a_j^t c_j \in \mathbb{R}^d$$

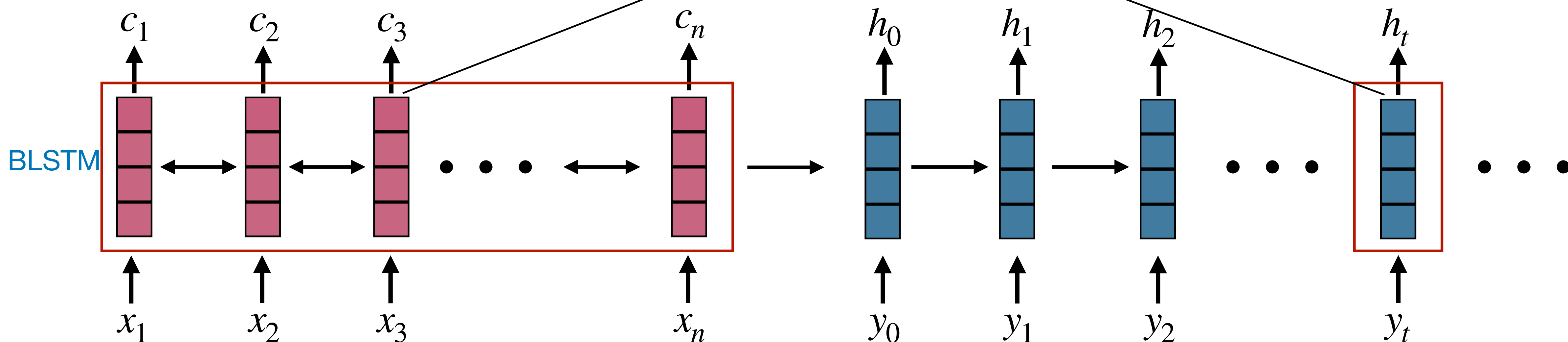
Attention output

$$a^t = \text{softmax}(e^t) \in (0,1)^n$$

Attention distribution

$$e_j^t = \text{sim}(c_j, h_t), \forall j \in \{1, \dots, n\}$$

Attention scores

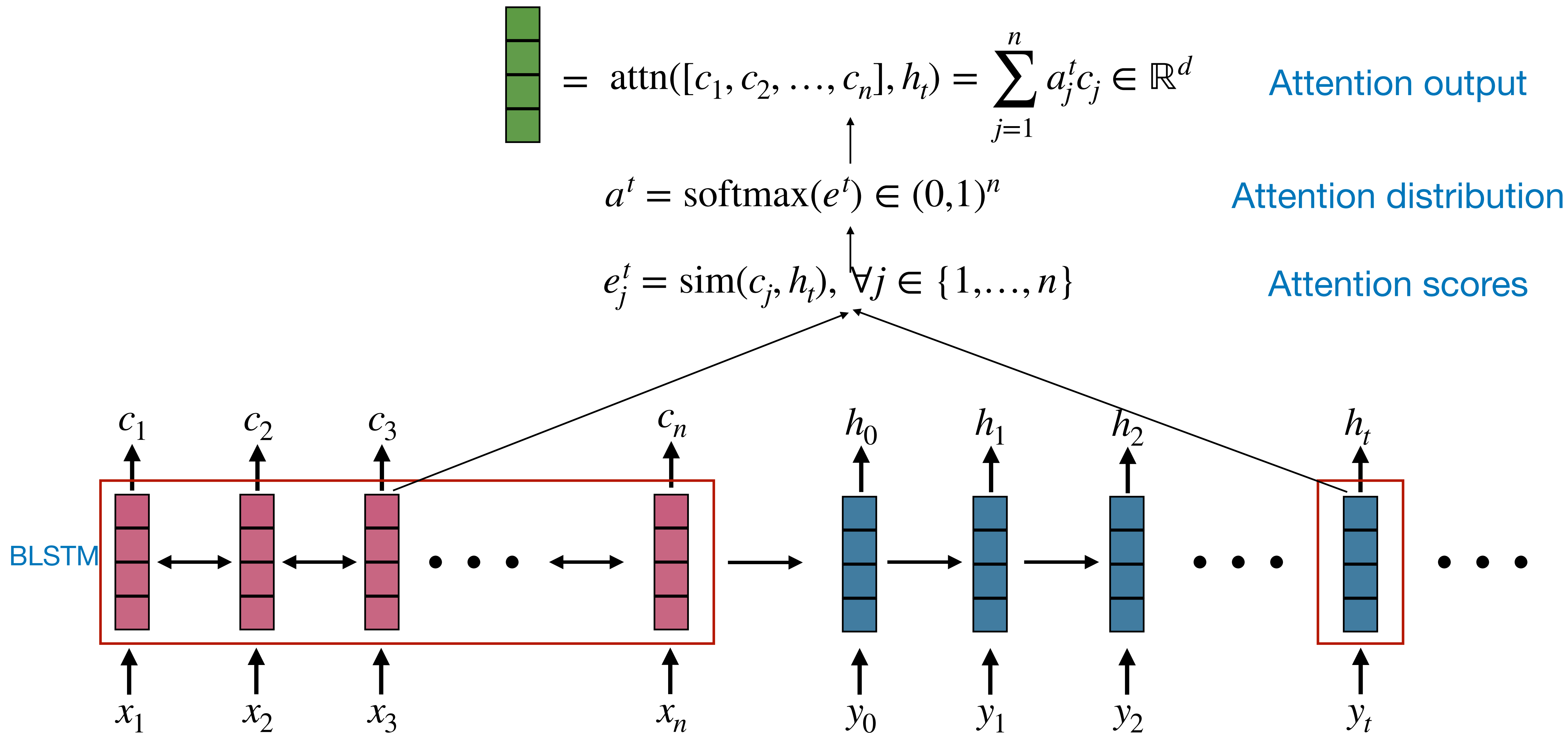


Softmax Function

$$e^t = [e_1^t, e_2^t, \dots, e_n^t]$$

$$\text{softmax}(e^t) = \left[\frac{\exp(e_1^t)}{\sum_{j=1}^n \exp(e_j^t)}, \frac{\exp(e_2^t)}{\sum_{j=1}^n \exp(e_j^t)}, \dots, \frac{\exp(e_n^t)}{\sum_{j=1}^n \exp(e_j^t)} \right]$$

Attention Mechanism



Types of Attention

- **Dot-product attention** (assumes equal dimensions for c and h)

$$\text{sim}(c_j, h_t) = c_j^T h_t$$

- **Multiplicative attention**

$$\text{sim}(c_j, h_t) = c_j^T W h_t, \text{ where } W \text{ is learnable weight matrix}$$

- **Additive attention**

$$\text{sim}(c_j, h_t) = v^T \tanh(W_c c_j + W_h h_t)$$

where W_c and W_h are learnable weight matrices and v is a learnable weight vector

Attention Improves Translation Performance

System	Ppl	BLEU
Winning WMT'14 system – <i>phrase-based</i> + <i>large LM</i> (Buck et al., 2014)		20.7
<i>Existing NMT systems</i>		
RNNsearch (Jean et al., 2015)		16.5
RNNsearch + unk replace (Jean et al., 2015)		19.0
RNNsearch + unk replace + large vocab + <i>ensemble</i> 8 models (Jean et al., 2015)		21.6
<i>Our NMT systems</i>		
Base	10.6	11.3
Base + reverse	9.9	12.6 (+1.3)
Base + reverse + dropout	8.1	14.0 (+1.4)
Base + reverse + dropout + global attention (<i>location</i>)	7.3	16.8 (+2.8)
Base + reverse + dropout + global attention (<i>location</i>) + feed input	6.4	18.1 (+1.3)
Base + reverse + dropout + local-p attention (<i>general</i>) + feed input	5.9	19.0 (+0.9)
Base + reverse + dropout + local-p attention (<i>general</i>) + feed input + unk replace		20.9 (+1.9)
<i>Ensemble</i> 8 models + unk replace		23.0 (+2.1)

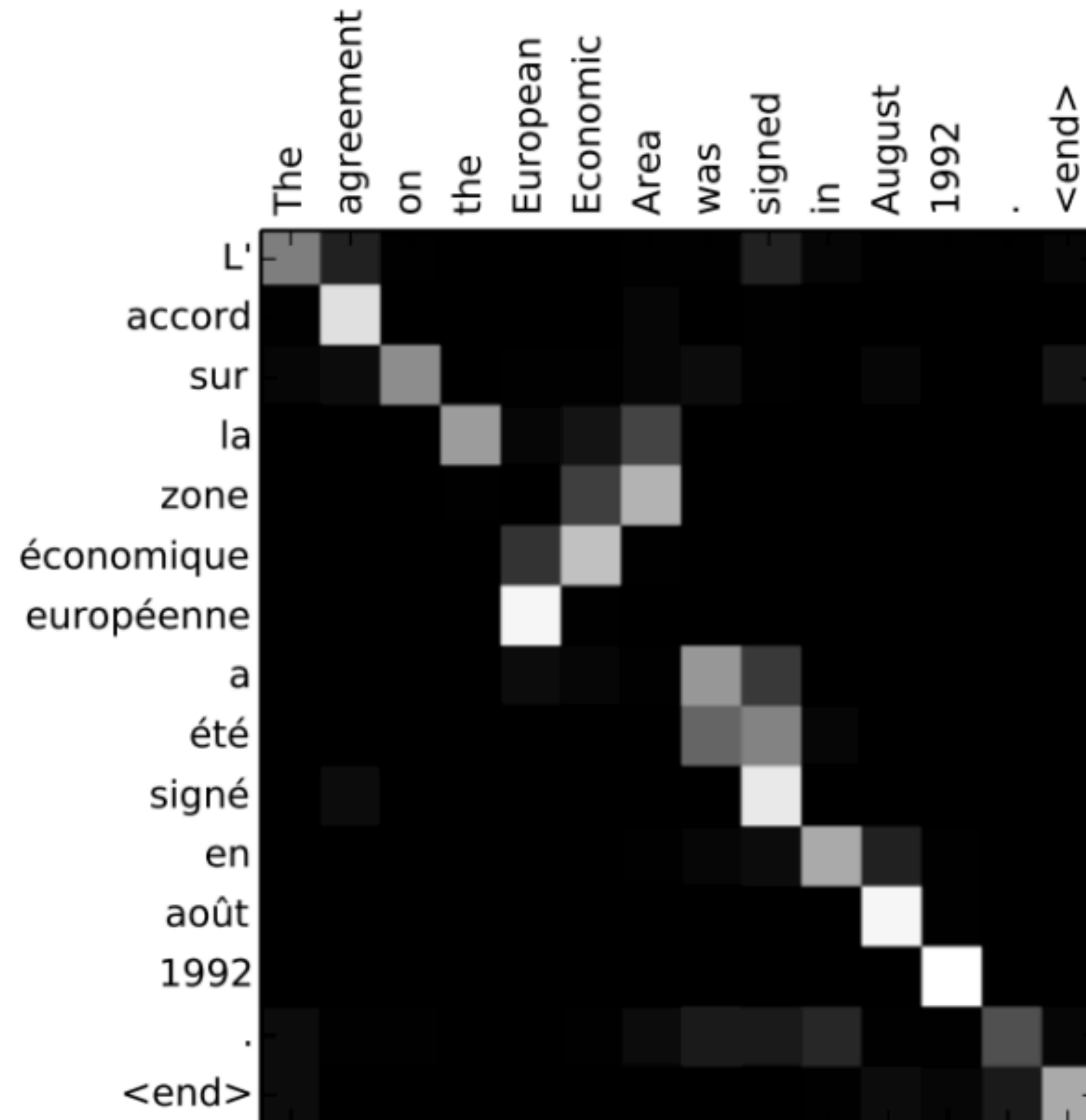
(Luong et al., 2015)

Google's Neural Machine Translation System: Bridging the Gap between Human and Machine Translation

Table 10: Mean of side-by-side scores on production data

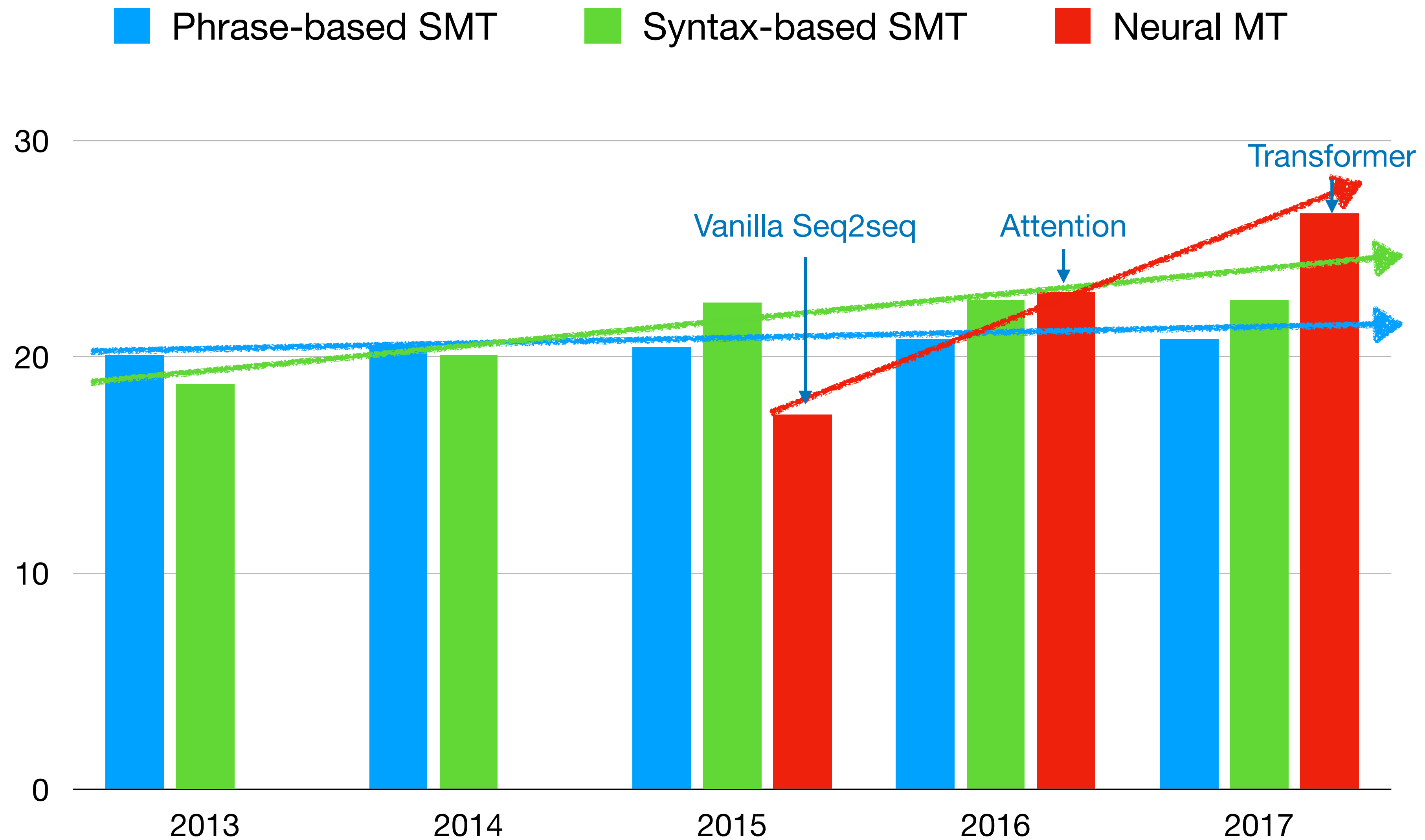
	PBMT	GNMT	Human	Relative Improvement
English → Spanish	4.885	5.428	5.504	87%
English → French	4.932	5.295	5.496	64%
English → Chinese	4.035	4.594	4.987	58%
Spanish → English	4.872	5.187	5.372	63%
French → English	5.046	5.343	5.404	83%
Chinese → English	3.694	4.263	4.636	60%

Visualizing Attention



Highly correlated with alignment

MT Progress



Reading Materials

- **Reading Materials**
 - Sequence to Sequence Learning with Neural Networks
 - Neural Machine Translation by Jointly Learning to Align and Translate